

```

#include <stdio.h>
#include <math.h>

// Function to find GCD
int gcd(int a, int b) {
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

// Function to find LCM
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

// Function to find hyperperiod
int find_hyperperiod(int p[], int n) {
    int h = p[0];
    for (int i = 1; i < n; i++) {
        h = lcm(h, p[i]);
    }
    return h;
}

int main() {
    int n;

    printf("Enter number of tasks: ");
    scanf("%d", &n);

    int C[n], P[n];

    printf("Enter computation time and period:\n");
    for (int i = 0; i < n; i++) {
        printf("Task %d (C P): ", i + 1);
        scanf("%d %d", &C[i], &P[i]);
    }

    // Calculate utilization
    float U = 0;
    for (int i = 0; i < n; i++) {

```

```

    U += (float)C[i] / P[i];
}

float bound = n * (pow(2, 1.0 / n) - 1);

printf("\nUtilization U = %.2f\n", U);
printf("Bound = %.2f\n", bound);

if (U <= bound)
    printf("Condition: %.2f <= %.2f (Schedulable)\n", U, bound);
else
    printf("Condition: %.2f > %.2f (Not Guaranteed)\n", U, bound);

// Sort tasks by period (Rate Monotonic priority)
for (int i = 0; i < n - 1; i++) {
    for (int j = i + 1; j < n; j++) {
        if (P[i] > P[j]) {
            int temp;

            temp = P[i]; P[i] = P[j]; P[j] = temp;
            temp = C[i]; C[i] = C[j]; C[j] = temp;
        }
    }
}

// Find hyperperiod
int hyper = find_hyperperiod(P, n);

int remaining[n];
for (int i = 0; i < n; i++)
    remaining[i] = 0;

int prev_task = -2;

printf("\n--- Compact Schedule ---\n");

for (int time = 0; time < hyper; time++) {

    // Release tasks
    for (int i = 0; i < n; i++) {
        if (time % P[i] == 0) {
            remaining[i] = C[i];
        }
    }
}

```

```

int current_task = -1;

// Pick highest priority task
for (int i = 0; i < n; i++) {
    if (remaining[i] > 0) {
        current_task = i;
        break;
    }
}

// Print only when task changes
if (current_task != prev_task) {
    if (current_task == -1)
        printf("Time %2d: Idle\n", time);
    else
        printf("Time %2d: Task %d\n", time, current_task + 1);
}

// Execute task
if (current_task != -1) {
    remaining[current_task]--;
}

prev_task = current_task;
}

return 0;
}

```

```

Enter number of tasks: 2
Enter computation time and period:
Task 1 (C P): 20 50
Task 2 (C P): 35 100

Utilization U = 0.75
Bound = 0.83
Condition: 0.75 <= 0.83 (Schedulable)

--- Compact Schedule ---
Time 0: Task 1
Time 20: Task 2
Time 50: Task 1
Time 70: Task 2
Time 75: Idle

Process returned 0 (0x0)   execution time : 13.904 s
Press any key to continue.

```

```
#include <stdio.h>
```

```
// Function to calculate GCD
```

```
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
```

```
// Function to calculate LCM
```

```
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
```

```
int main() {
    int n;
```

```
    printf("Enter number of tasks: ");
    scanf("%d", &n);
```

```
    int burst[n], period[n], deadline[n];
    int remaining[n], next_arrival[n], absolute_deadline[n];
```

```
    // Input task details
    for (int i = 0; i < n; i++) {
```

```

printf("Enter burst time, period, and relative deadline for task %d: ", i + 1);
scanf("%d %d %d", &burst[i], &period[i], &deadline[i]);

remaining[i] = 0;
next_arrival[i] = 0;
absolute_deadline[i] = 0;
}

// Display task table
printf("\n%-10s %-10s %-10s %-10s\n", "Task", "Burst", "Period", "Deadline");
for (int i = 0; i < n; i++) {
    printf("%-10d %-10d %-10d %-10d\n", i + 1, burst[i], period[i], deadline[i]);
}

// Calculate hyperperiod
int hyperperiod = period[0];
for (int i = 1; i < n; i++) {
    hyperperiod = lcm(hyperperiod, period[i]);
}

printf("\n--- Preemptive EDF Scheduling ---\n");
printf("%-10s %-20s\n", "Time", "Executing Task");

// EDF Scheduling
for (int time = 0; time < hyperperiod; time++) {

    // Check for task arrivals
    for (int i = 0; i < n; i++) {
        if (time == next_arrival[i]) {
            remaining[i] = burst[i];
            absolute_deadline[i] = time + deadline[i];
            next_arrival[i] += period[i];
        }
    }
}

int min_deadline = 1000000000;
int selected = -1;

// Select task with earliest deadline
for (int i = 0; i < n; i++) {
    if (remaining[i] > 0 && absolute_deadline[i] < min_deadline) {
        min_deadline = absolute_deadline[i];
        selected = i;
    }
}

```

```
}

// Execute task
if (selected == -1) {
    printf("%-10d %-20s\n", time, "CPU Idle");
} else {
    remaining[selected]--;
    printf("%-10d Task %d\n", time, selected + 1);
}
}

return 0;
}
```

```
Enter number of tasks: 3
Enter burst time, period, and relative deadline for task 1: 3
20
7
Enter burst time, period, and relative deadline for task 2: 2
5
4
Enter burst time, period, and relative deadline for task 3: 2
10
8
```

Task	Burst	Period	Deadline
1	3	20	7
2	2	5	4
3	2	10	8

--- Preemptive EDF Scheduling ---

Time(ms)	Executing Task
0	2
1	2
2	1
3	1
4	1
5	3
6	3
7	2
8	2
9	CPU Idle
10	2
11	2
12	3
13	3
14	CPU Idle
15	2
16	2
17	CPU Idle
18	CPU Idle
19	CPU Idle

Process returned 0 (0x0) execution time : 45.474 s
Press any key to continue.